

TEST PLAN

BlazeDemo Flight Booking Automation Framework

Selenium WebDriver · TestNG · Maven · ExtentReports · Jenkins · Docker

Prepared By: Ayush Sharma

Github: https://github.com/AYUSH-YL/Capstone_project

Site: <https://blazedemo.com/reserve.php>

1. INTRODUCTION

The purpose of this Master Test Plan is to define the testing scope, strategy, and execution workflow for the BlazeDemo flight booking application. The automation framework is engineered to run functional and security tests, verifying that users can search for active flight schedules, complete traveler details, bypass payment gateway constraints using mock systems, and receive valid booking confirmation details.

This plan also outlines how the automated scripts gracefully handle bad test data and run continuously inside a Jenkins CI/CD pipeline using Docker containers.

2. TEST ITEMS

We are testing the core transactional pages and user input forms of the BlazeDemo application:

1. Landing (Search) Page:

- Departure City drop-down selection list (`fromPort`)
- Destination City drop-down selection list (`toPort`)
- "Find Flights" submission button

2. Flight Selection Page:

- Flight options availability grid
- "Choose This Flight" triggers

3. Purchase Page:

- Passenger Name input field (`inputName`)
- Location details (Address, City, State, Zip Code)
- Card Type selection drop-down (`cardType`)
- Payment details (Credit Card Number, Month, Year, Name on Card)
- "Purchase Flight" submission button

4. Confirmation Page:

- Final Booking Success Message ("Thank you")
- Transaction Invoice details (Purchase ID, Status, Booking Date, and Card Number mask)

5. Security & Portal Forms:

- Registration Form input fields (Name, Company, Email, Password, Password Confirm)
- Login Form input fields (Email, Password)

3. FEATURES TO BE TESTED

The automation scripts are split into two major test classes, mapped directly to your source files:

Module A: Web Portal Security & Exception Sweeps (`LoginTest.java`)

- **Unauthenticated Registration Sweep:** Testing the registration page with invalid data arrays to confirm the system rejects unauthorized setups.
- **HTTP State Check:** Parsing the page source DOM to verify the system handles expired workflows by redirecting users to a secure 419 Page Expired warning page.
- **Bypass Prevention:** Injecting raw login attempts with invalid credentials to ensure application access is blocked.
- **History State Recovery:** Triggering browser-back operations (`driver.navigate().back()`) to confirm the driver reference does not crash and recovers back onto the main page safely.
- **Automatic Failure Screen Capture:** Capturing high-resolution browser screenshots (`Registration_419_Error_Page.png` and `Login_419_Error_Page.png`) at key failure states.

Module B: Data-Driven Flight Booking Loop (`BlazeDemoTest.java`)

- **Excel-Driven Search:** Reading dynamic departure and destination configurations from the `TestData.xlsx` sheet.
- **Result Table Verification:** Inspecting the results table layout to confirm it has active flight options.
- **Special Character Resiliency:** Verifying that the passenger info fields do not crash when processing names containing symbols (e.g., `John##Doe!!` or `Amit_Sharma_$$`).

- **Code Injection Protection:** Testing input security by trying to submit HTML scripts (e.g., `<Admin>`) to make sure the app handles text safely.
- **Self-Healing Drop-down Fallback:** If the external Excel sheet contains unsupported payment options (e.g., *Mastercard*), our fallback engine captures the exception, logs a warning, defaults the dropdown to *Visa*, and keeps the script running smoothly without a crash.
- **Completed Booking Assertion:** Checking that the checkout lifecycle ends successfully by asserting the keyword string "Thank you" on the confirmation screen.

4. FEATURES NOT TO BE TESTED

- **Real-world Bank Transactions:** No live card check, credit limit sweeps, or real currency processing (we use mock sandbox values).
- **Database Storage Checks:** Verification of long-term database storage (the mock server processes booking entries temporary and in-flight).
- **Static Visual Components:** Checking individual pixel layouts, copyright text lines, top branding logos, and static main navigation link bars.
- **Recovery Email Dispatches:** Real SMTP email routing checks or external inbox verifications when triggering password reset forms.
- **Config Variable Edits at Runtime:** Live changes to base settings (like explicit timeouts or the base URL) while the TestNG suite is executing.

5. TESTING APPROACH

We are using Automated Functional Testing, structured around the **Page Object Model (POM)**:

- **Black Box Testing:** Verifying standard end-to-end user workflows (searching, selecting, filling forms, and buying).
- **Grey Box Verification:** Validating that data written inside our external Excel test files (`TestData.xlsx`) matches the information processed through UI elements on the web browser.
- **Automation Framework (POM):**
 - Separate page classes store web element locations (locators) and actions, making scripts easier to maintain.
 - **TestNG** controls our test priorities and sequence order (`priority = 1`, `dependsOnMethods`).

- **Apache POI** reads dynamic inputs from Excel rows.
- **Extent Reports** generates structured visual reports.

6. ITEM PASS/FAIL CRITERIA

- **Total Automated Test Cases:** 4 Main Test Sequences
- **Pass Criteria:** A test passes if it completes all automated actions, encounters no unhandled exceptions, and finds its target success assertion (e.g., confirming the "Thank you" message).
- **Fail Criteria:** A test fails if it hits an unexpected script exception, fails an assertion, or prevents remaining rows in the Excel data sheet from running.
- **Acceptance Metric:** Minimum **100% Pass Rate** on all functional flight booking regression cases.

7. SUSPENSION CRITERIA AND RESUMPTION REQUIREMENTS

Suspension (Pause) Conditions:

- The BlazeDemo target website is completely down or unreachable.
- The GitHub repository experiences access failures or code syncing issues.
- Massive local network dropouts block access to testing tools or servers.

Resumption (Restart) Conditions:

- BlazeDemo servers or local internet connections return to a normal active state.
- A quick manual page-load check confirms that target endpoints are active and accessible.

8. TEST DELIVERABLES

1. **Automation Source Code:** Java-based POM automation scripts (Page classes & Test scripts).
2. **Test Orchestration Suite:** `testng.xml` script configuration.
3. **Dynamic Test Data:** `TestData.xlsx` sheet containing test cases and inputs.
4. **Visual Execution Logs:** `ExtentReport.html` dashboard showing charts and run results.
5. **Execution Proof:** Captured screenshot files of errors and confirmation receipts.

- 6. **Virtualization & CI Files:** Dockerfile and Jenkinsfile configurations.

9. ENVIRONMENTAL NEEDS

- **Workstation Hardware:** Local or Virtual PC with a minimum of 8GB RAM.
- **Software Languages & Runtimes:** Java JDK 17, Apache Maven 3.x.
- **Development IDE:** Eclipse IDE or IntelliJ IDEA.
- **Web Driver Engine:** Selenium Java 4.x Client Library.
- **Target Web Browsers:** Google Chrome, Mozilla Firefox, and Microsoft Edge.
- **CI/CD & Containers:** Docker Desktop (containerization) and Jenkins (local automated server).

10. STAFFING AND TRAINING NEEDS

- **Assigned Automation Lead:** Ayush Sharma
- **Team Capacity:** 1 dedicated QA Automation Engineer.
- **Onboarding & Training:** No additional training required; the resource is fully qualified in POM frameworks, Selenium WebDriver, TestNG, Docker, and Jenkins pipelines.

11. SCHEDULE

The automation project has been completed according to the following 7-day milestone schedule:

Day	Milestone Activity	Actual Status
Day 1	Requirement Analysis & Master Test Plan Preparation	Completed
Day 2	Test Case Design & External Test Data Mapping	Completed

Day	Milestone Activity	Actual Status
Day 3	Maven Framework Setup, POM Architecture & dependencies configuration	Completed
Day 4	Page Object Classes development (Locators & action methods)	Completed
Day 5	Test Script development (incorporating security sweeps and booking loops)	Completed
Day 6	ExtentReports implementation, Docker and Jenkins integration setup	Completed
Day 7	Execution, Defect Analysis, HTML reporting, and Final Project Sign-off	Completed

13. RISKS AND CONTINGENCIES

- **Risk:** Automated headlessly-triggered tests crashing inside headless Linux Docker environments due to missing Chrome options or system layouts.
- **Contingency Fix:** Programmed an automatic environment-sensing switch in `BaseTest.java`. When running inside Docker containers, the framework turns on performance configurations (`--headless=new`, `--no-sandbox`, and `--disable-dev-shm-usage`) to guarantee clean, background browser execution.

14. APPROVALS

Trainer / Evaluator Review Required: Yes.